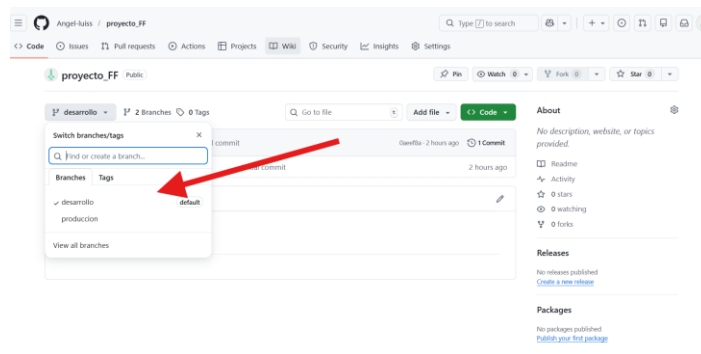


Prueba técnica full stack desarrollada por Luis Angel Tucubal

Paso 1) Crear repo + ramas (Git)

- git init
- git branch -M desarrollo
- git remote add origin
- git push -u origin desarrollo
- git checkout -b produccion
- git push -u origin produccion
- git checkout desarrollo



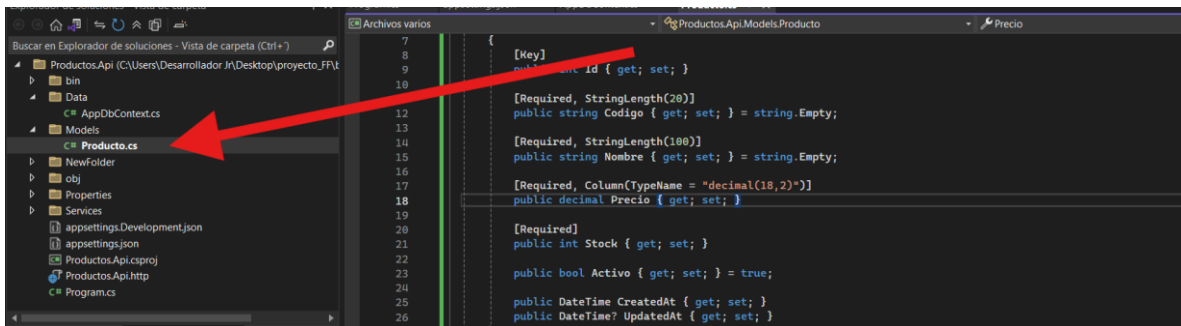
Paos 2) Inicialización del Backend (.NET 8)

Se creo una Api usando

dotnet new webapi -n Productos.Api --framework net8.0

Creamos las carpetas:

- Models
- Data
- Services



Paso 3) Configuración de Base de Datos y EF Core

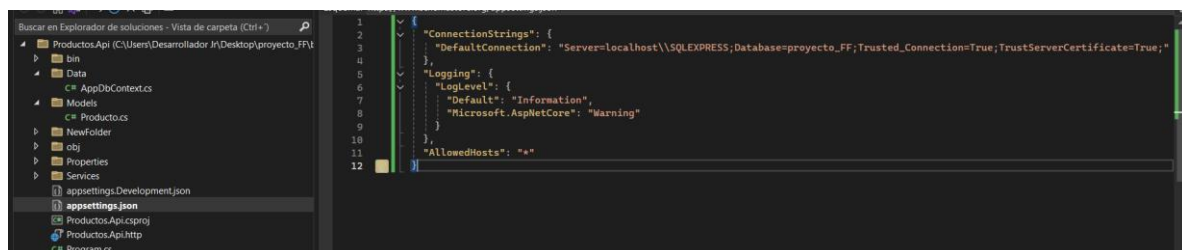
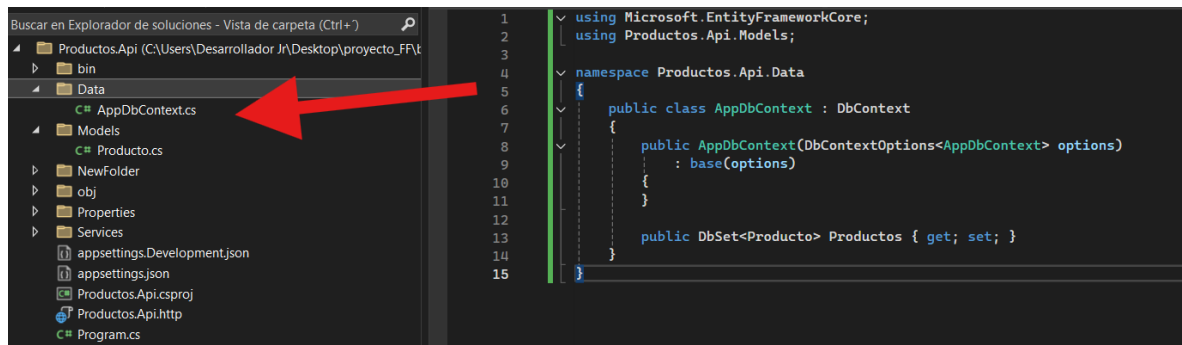
1. Se instaló **SQL Server Express (SQLEXPRESS)**.
2. Se creó la base de datos **proyecto_FF** en SSMS.
3. Se configuró la cadena de conexión en appsettings.json usando Windows Authentication:

"ConnectionStrings": {

"DefaultConnection":

"Server=localhost\\SQLEXPRESS;Database=proyecto_FF;Trusted_Connection=True;TrustServerCertificate=True;" }

4. Se registró el AppDbContext en Program.cs para conectar la API con SQL Server mediante Entity Framework Core.
5. Se verificó que el proyecto compilara correctamente con dotnet build.



Paso 4) Configuración de Base de Datos

Crear la base de datos:

```
CREATE DATABASE proyecto_FF;
```

```
CREATE LOGIN proyecto_FF WITH PASSWORD = 'proyecto_FF';
```

```
GO
```

```
USE proyecto_FF;
```

```
GO
```

```
CREATE USER proyecto_FF FOR LOGIN proyecto_FF;
```

```
GO
```

```
ALTER ROLE db_owner ADD MEMBER proyecto_FF;
```

```
GO
```

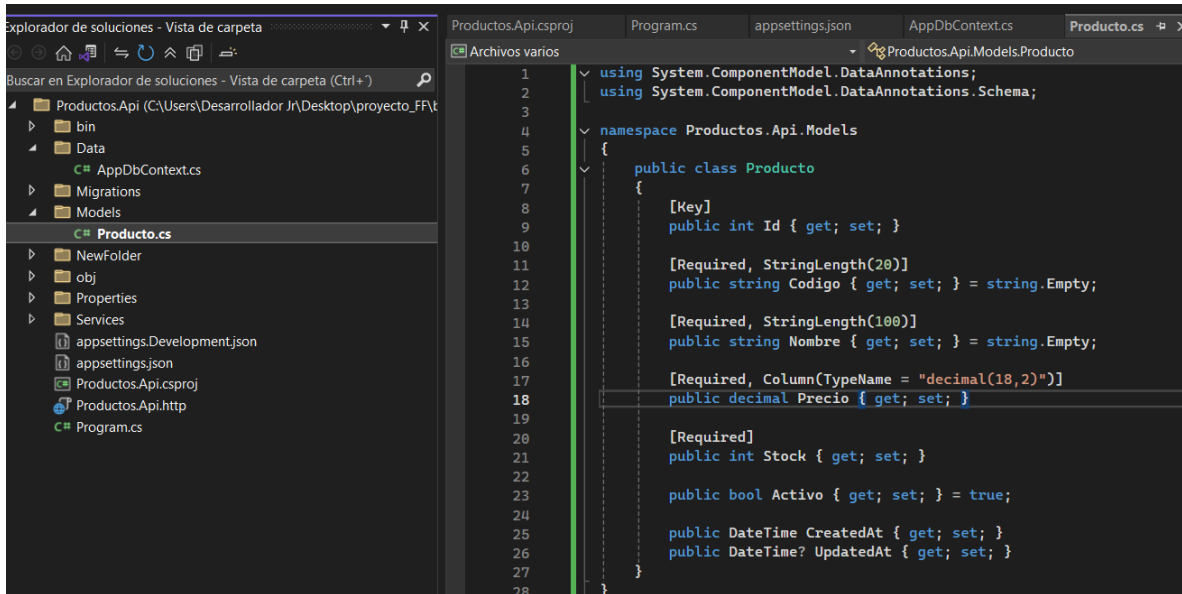
2. Configuración de la cadena de conexión

En el archivo appsettings.json:

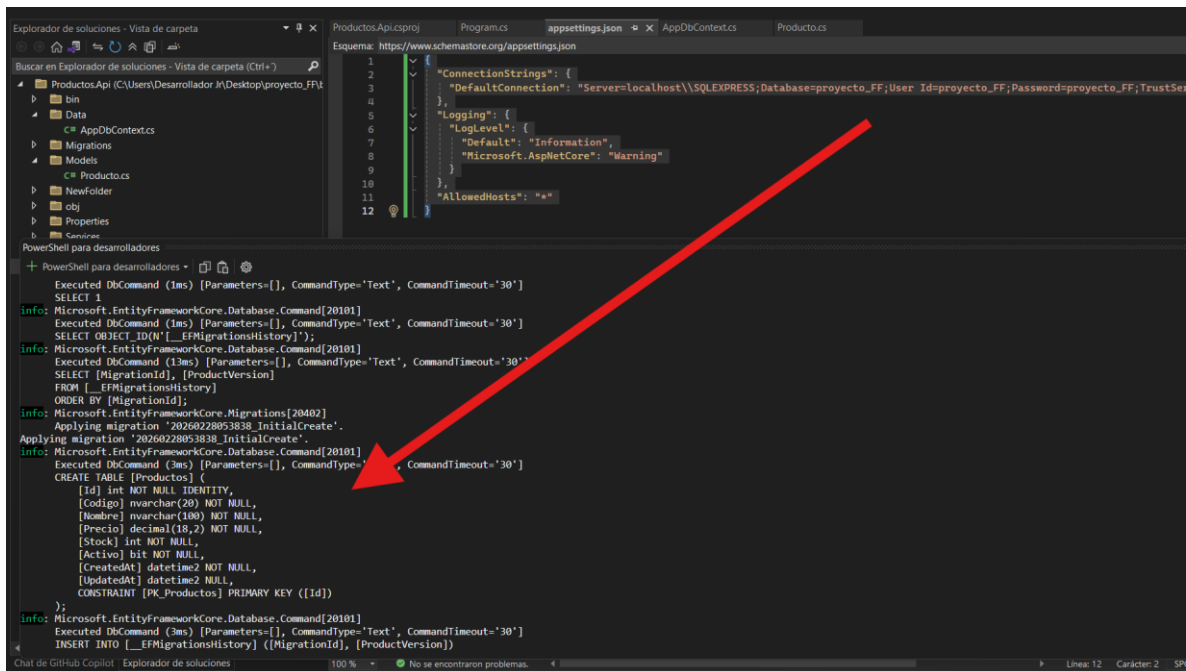
```
{  
  "ConnectionStrings": {  
    "DefaultConnection":  
    "Server=localhost\\SQLEXPRESS;Database=proyecto_FF;User  
Id=proyecto_FF;Password=proyecto_FF;TrustServerCertificate=True;"  
  }  
}
```

Paso 5) Aplicar migraciones

Desde la carpeta del proyecto (Productos.Api):



dotnet ef database update



Paso 6) Implementamos un CRUD completo en ASP.NET Core

Creamos un **Controller** llamado: ProductosController

Método	Endpoint	Función
GET	/api/Productos	Listar productos
GET	/api/Productos/{id}	Obtener por ID
POST	/api/Productos	Crear producto
PUT	/api/Productos/{id}	Actualizar producto
DELETE	/api/Productos/{id}	Eliminar producto

Conectamos el Controller con Entity Framework

Inyectamos el contexto: `private readonly ApplicationDbContext _context;`

Probamos la API usando Swagger

Swagger nos permitió:

- Enviar un POST con JSON
- Verificar que el registro se guardó
- Ejecutar un GET para confirmar que el producto estaba en la base

- **GET /api/Productos**

Permite listar todos los productos registrados en la base de datos.

Soporta filtros opcionales por:

- codigo
- nombre
- activo

También incluye un endpoint para obtener un producto específico por su Id.

Productos

GET /api/Productos

Parameters

Name	Description
codigo	string (query)
nombre	string (query)
activo	boolean (query)

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:3030/api/Productos' \
  -H 'accept: text/plain'
```

Request URL

http://localhost:3030/api/Productos

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:3030/api/Productos' \
  -H 'accept: text/plain'
```

Request URL

http://localhost:3030/api/Productos

Server response

Code Details

200

Response body

```
{
  "id": 1,
  "codigo": "P001",
  "nombre": "Producto A",
  "precio": 100,
  "stock": 10,
  "activo": true,
  "createdAt": "2024-02-28T01:45:12.000Z",
  "updatedAt": null
},
{
  "id": 2,
  "codigo": "P002",
  "nombre": "Producto B",
  "precio": 200,
  "stock": 5,
  "activo": true,
  "createdAt": "2024-02-28T01:45:12.000Z",
  "updatedAt": null
}
```

Response headers

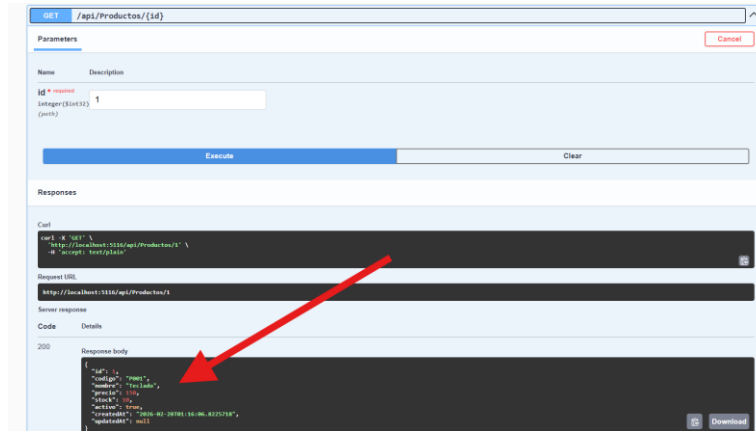
```
content-type: application/json; charset=utf-8
date: Sat, 28 Feb 2026 00:27:54 GMT
server: Node.js
transfer-encoding: chunked
```

Responses

- **GET /api/Productos/{id}**

Obtiene un producto específico por su Id.

Si el producto no existe (o está inactivo, según la lógica aplicada), retorna 404 Not Found.



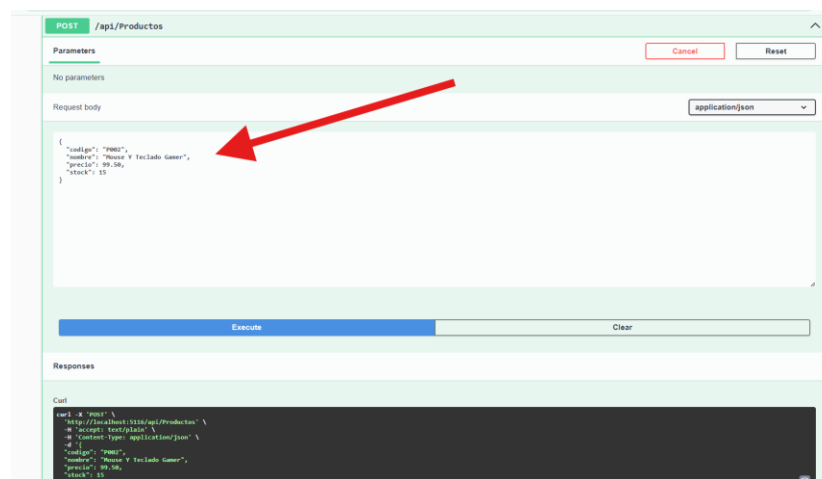
- **POST /api/Productos**

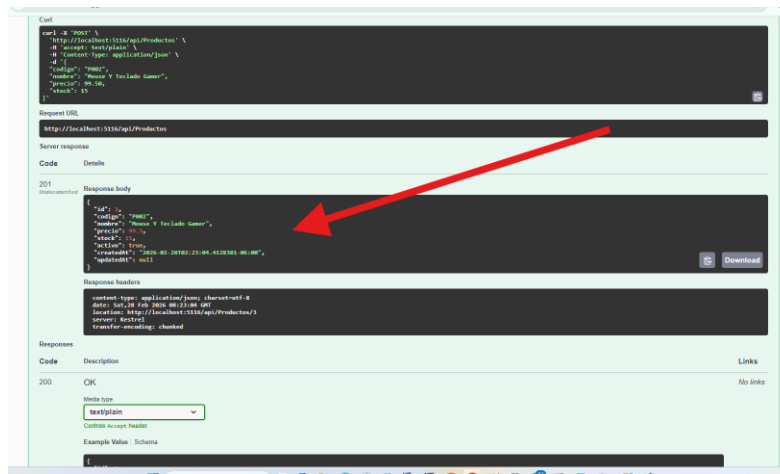
Permite crear un nuevo producto.

Al momento de la creación:

- Se asigna automáticamente CreatedAt
- Se establece Activo = true por defecto

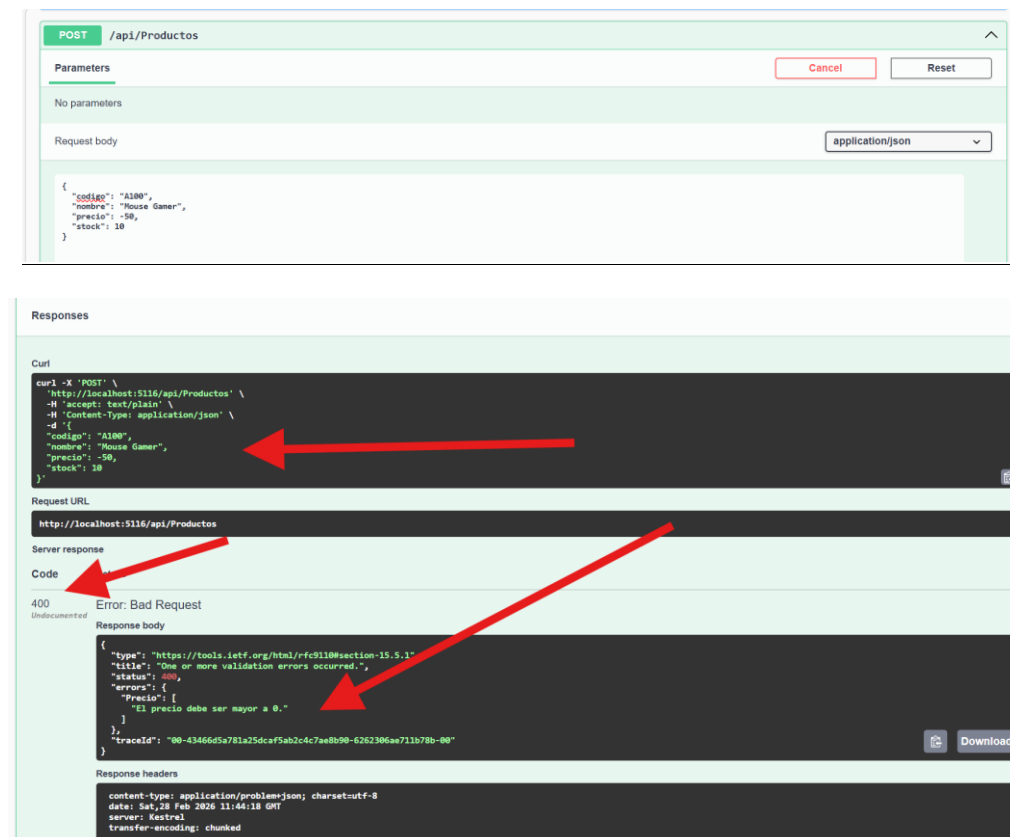
Devuelve código **201 Created** cuando el registro se guarda correctamente.





Prueba: Precio negativo (POST)

Se envió un producto con el campo precio en valor negativo. La API rechazó la solicitud y devolvió 400 Bad Request, confirmando que la validación impide registrar precios menores a cero.



Prueba: Stock negativo (POST)

Se envió un producto con el campo stock en valor negativo. La API devolvió 400 Bad Request, verificando que no se permiten existencias menores a cero.

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** /api/Productos
- Parameters:** No parameters
- Request body:**

```
{
  "codigo": "A101",
  "nombre": "Teclado",
  "precio": 150,
  "stock": -3
}
```
- Response:** 400 Bad Request
- Response body:**

```
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.1",
  "title": "One or more validation errors occurred.",
  "status": 400,
  "errors": {
    "stock": [
      "El stock no puede ser negativo."
    ]
  },
  "traceId": "00-f033d061c044edSeed7699d38795a71-6d793af6936118fa3-00"
}
```

Red arrows point to the `stock: -3` in the request body, the `400` status code, and the error message in the response body.

Prueba: Código repetido (POST)

Se intentó registrar un producto con un **código** ya existente. La API devolvió **400 Bad Request** con el mensaje **“El código ya existe.”**, demostrando que la restricción **UNIQUE** se está aplicando

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** /api/Productos
- Parameters:** No parameters
- Request body:**

```
{
  "codigo": "A200",
  "nombre": "MonitorGH",
  "precio": 1200,
  "stock": 5
}
```
- Response:** 201
- Response body:**

```
{
  "id": 4,
  "codigo": "A200",
  "nombre": "MonitorGH",
  "precio": 1200,
  "stock": 5,
  "activo": true,
  "createdAt": "2026-02-28T05:52:58.3100745-06:00",
  "updatedAt": null
}
```

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:5116/api/Productos' \
  -H 'accept: text/plain' \
  -H 'Content-Type: application/json' \
  -d '{
    "codigo": "A200",
    "nombre": "MonitorGM",
    "precio": 1200,
    "stock": 5
  }'
```

Request URL

http://localhost:5116/api/Productos

Server response

Code

400

Error: Bad Request

Response body

El código ya existe.

Response headers

```
content-type: text/plain; charset=utf-8
date: Sat, 28 Feb 2026 12:09:46 GMT
server: Kestrel
transfer-encoding: chunked
```

Code	Description	Links
200	OK	No links

Prueba: Código repetido (PUT)

Se intentó actualizar un producto activo asignándole un **código** que ya pertenece a otro producto. La API devolvió **400 Bad Request** con el mensaje “**El código ya existe.**”, confirmando que la validación también funciona al editar registros.

Execute Clear

Responses

Curl

```
curl -X 'PUT' \
  'http://localhost:5116/api/Productos/2' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": 2,
    "codigo": "A200",
    "nombre": "Teclado Editado",
    "precio": 150,
    "stock": 10,
    "activo": true
  }'
```

Request URL

http://localhost:5116/api/Productos/2

Server response

Code

400

Error: Bad Request

Response body

El código ya existe.

Response headers

```
content-type: text/plain; charset=utf-8
date: Sat, 28 Feb 2026 12:17:30 GMT
```

- **PUT /api/Productos/{id}**

Permite actualizar un producto existente.

- Se valida que el Id coincida
- Se asigna automáticamente UpdatedAt
- Devuelve **204 No Content** cuando la actualización es exitosa

PUT /api/Productos/{id}

Parameters

Name	Description
id * required	
integer(\$int32)	1
(path)	

Request body

application/json

```
{  "id": 1,  "codigo": "P0001",  "nombre": "Futbolito Gamer",  "precio": 250,  "stock": 8,  "activo": true,  "createdAt": "2020-02-28T00:00:00",  "updatedAt": null}
```

Execute Clear

Responses

Curl

```
curl -X 'PUT' \  http://localhost:5156/api/Productos/1 \  -H 'accept: */*' \  -H 'content-type: application/json' \  -d '{  "id": 1,  "codigo": "P0001",  "nombre": "Futbolito Gamer",  "precio": 250,  "stock": 8,  "activo": true,  "createdAt": "2020-02-28T00:00:00",  "updatedAt": null  }'
```

Request URL

http://localhost:5156/api/Productos/1

Server response

Code	Details
204	

Response headers

```
date: Sat, 28 Feb 2020 07:40:56 GMT  server: Restful3
```

Responses

Code	Description	Links
200	OK	No links

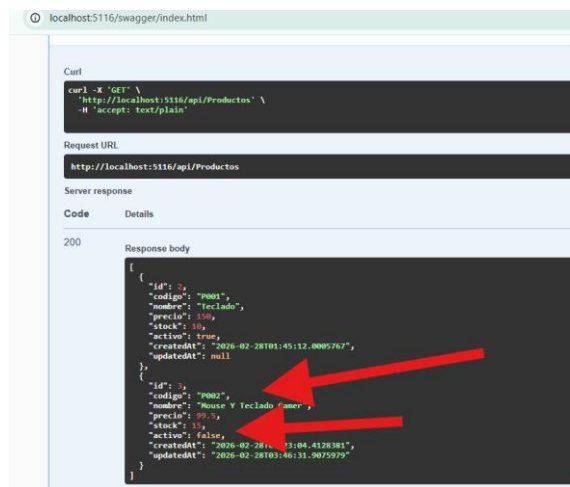
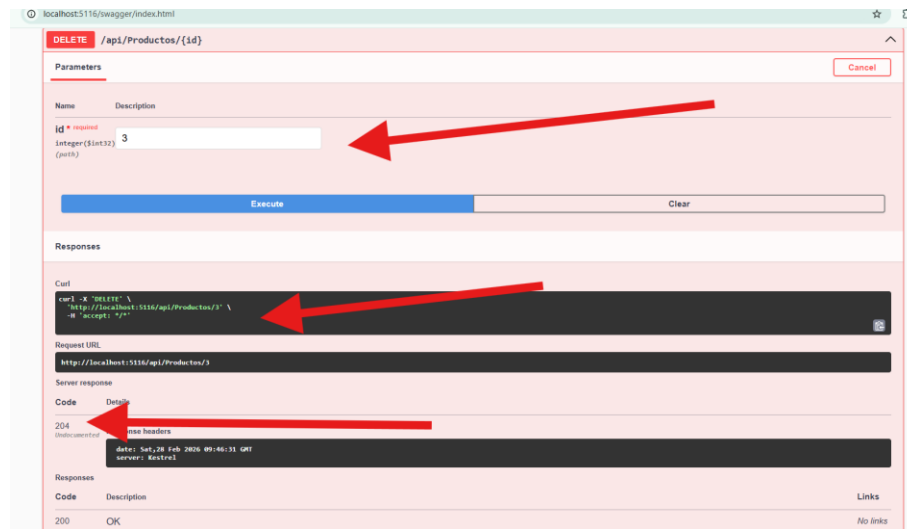
- **DELETE /api/Productos/{id}**

Implementado como **Soft Delete**.

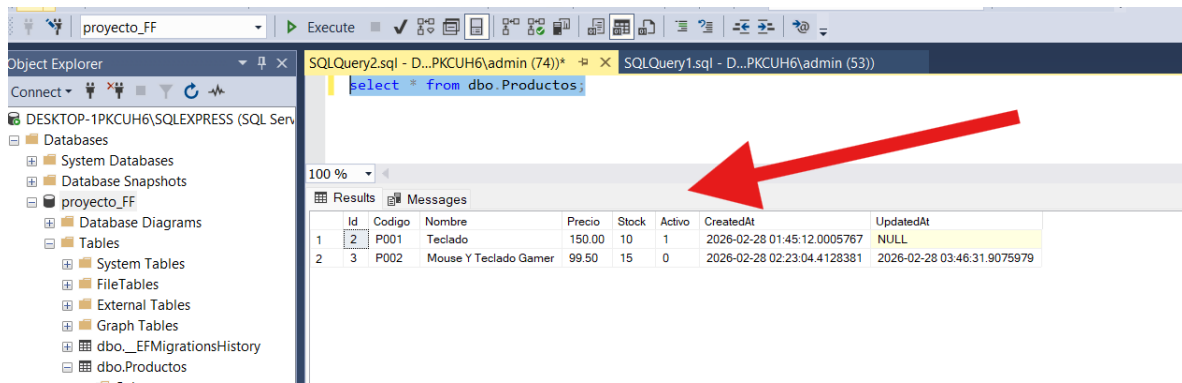
En lugar de eliminar físicamente el registro:

- Se cambia Activo = false
- Se actualiza UpdatedAt

Devuelve **204 No Content** si la operación es correcta.

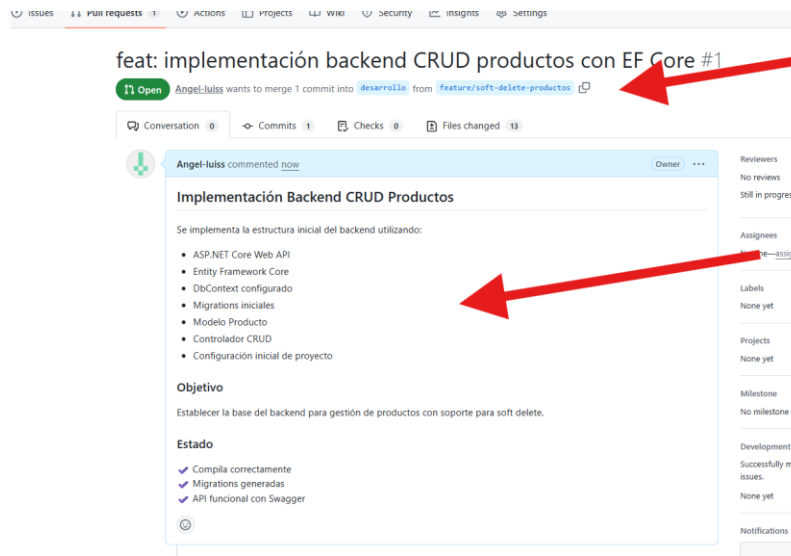


Conexión exitosa ASP.NET Core con SQL Server

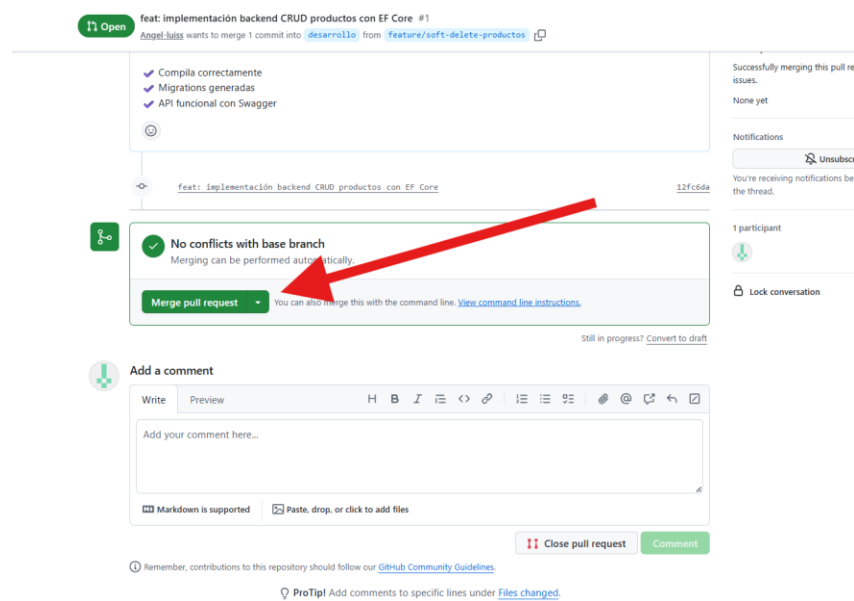


Crea el Pull Request

Durante el desarrollo del backend se aplicó un flujo de trabajo profesional utilizando Git. Se trabajó en una rama feature/soft-delete-productos, donde se realizaron los cambios correspondientes y se confirmaron mediante commits estructurados.



Posteriormente, la rama fue subida al repositorio remoto en GitHub y se creó un Pull Request hacia la rama desarrollo. El sistema validó que no existían conflictos, permitiendo realizar el merge automáticamente.



Finalmente, el Pull Request fue aprobado y fusionado correctamente, cerrándose y eliminándose la rama feature para mantener el repositorio organizado.

Este proceso garantizó un control de versiones ordenado, seguro y alineado con buenas prácticas de desarrollo colaborativo.

Merged

feat: implementación backend CRUD productos con EF Core #1

Angel-luiss merged 1 commit into desarrollo from feature/soft-delete-productos

Estado

✓ Compila correctamente

✓ Migrations generadas

✓ API funcional con Swagger

🕒

feat: implementación backend CRUD productos con EF Core

12fc6da

🔄

Angel-luiss merged commit 7621b22 into desarrollo now

Revert

🔗

Pull request successfully merged and closed

You're all set — the feature/soft-delete-productos branch can be safely deleted.

Delete branch

👤

Add a comment

Write

Preview

H B I

Add your comment here...

📄 Markdown is supported

Paste, drop, or click to add files

Comment

Development

Successfully merging this pull request issues.

None yet

Notifications

🔔 Unsubscribe

You're receiving notifications because the open/close state.

1 participant

🔒 Lock conversation

📌 Remember, contributions to this repository should follow our [GitHub Community Guidelines](#).

🔗 ProTip!

Add comments to specific lines under [Files changed](#)

Creación del Pull Request

En esta etapa se creó un Pull Request desde la rama feature/backend-service hacia la rama desarrollo.

Se verificó que no existieran conflictos y se documentaron los cambios realizados, incluyendo la implementación de la capa Service y las validaciones.

Open a pull request

Create a new pull request by comparing changes across two branches. If you need to, you can also [compare across forks](#). [Learn more about diff comparisons](#)

base: desarrollo ← compare: feature/backend-service ✓ Able to merge. These branches can be automatically merged.

Add a title *

refactor: implementación capa Service y validaciones en productos

Add a description

Write Preview

Refactor Arquitectónico - Capa Service

Se implementa una capa Service para desacoplar la lógica de negocio del controlador.

Cambios realizados:

- Creación de IProductoService
- Implementación de ProductoService
- Migración para validaciones y restricción única en Código
- Ajustes en Controller para utilizar Service
- Configuración de dependencias en Program.cs

Objetivo

Mejorar la arquitectura aplicando separación de responsabilidades (Controller → Service → Data)

Markdown is supported Paste, drop, or click to add files

Create pull request

Remember, contributions to this repository should follow our [GitHub Community Guidelines](#)

Verificación de conflictos

GitHub confirmó que no existían conflictos con la rama base, indicando que el merge podía realizarse automáticamente.

Esto valida que la rama estaba correctamente sincronizada con desarrollo.

Angel-luiss commented now

Owner

Refactor Arquitectónico - Capa Service

Se implementa una capa Service para desacoplar la lógica de negocio del controlador.

Cambios realizados:

- Creación de IProductoService
- Implementación de ProductoService
- Migración para validaciones y restricción única en Código
- Ajustes en Controller para utilizar Service
- Configuración de dependencias en Program.cs

Objetivo

Mejorar la arquitectura aplicando separación de responsabilidades (Controller → Service → Data).

refactor: implementación capa Service y validaciones en productos

b5c8b02

No conflicts with base branch

Merging can be performed automatically

Merge pull request

You can also merge this with the command line. [View command line instructions](#)

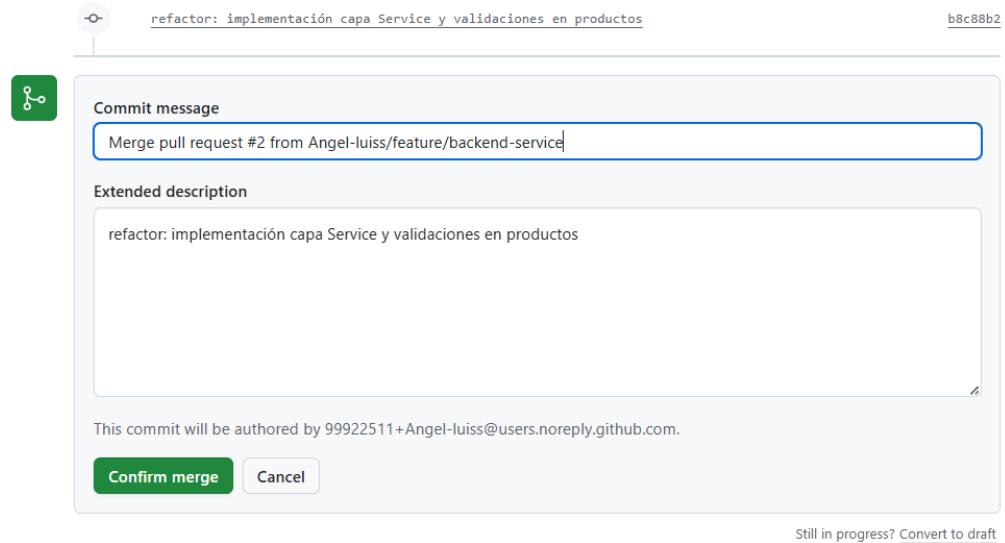
Still in progress? Convert to draft

1 participant

Confirmación del Merge

Se procedió a realizar el Merge Pull Request, confirmando la integración de los cambios en la rama desarrollo.

El commit final consolidó la implementación de la capa Service dentro del flujo principal del proyecto.

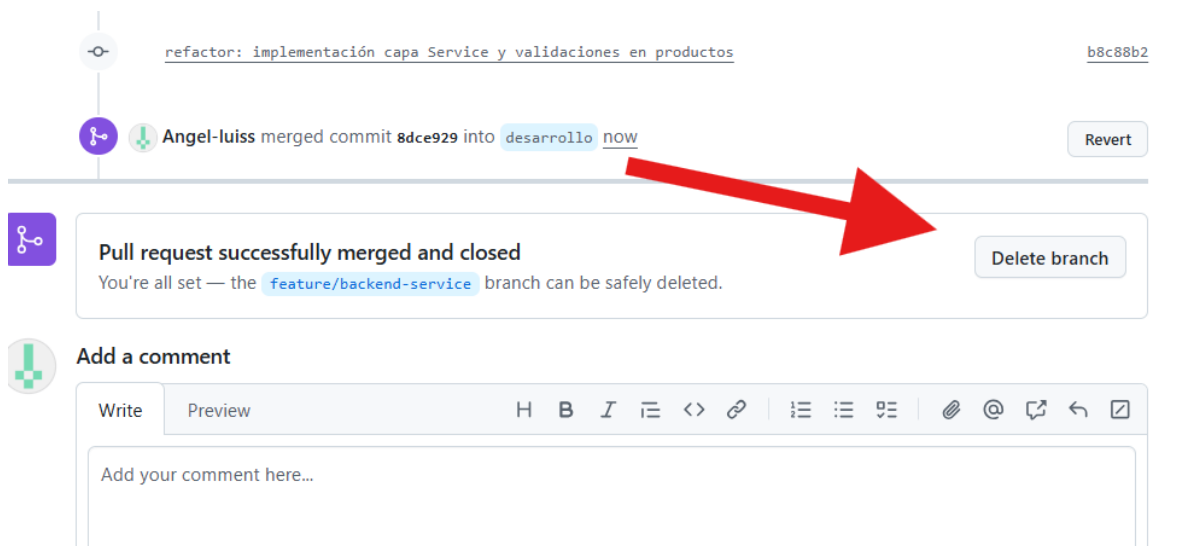


The screenshot shows the 'Confirm merge' dialog in GitHub. At the top, the branch name 'refactor: implementación capa Service y validaciones en productos' and commit hash 'b8c88b2' are visible. The dialog has a green 'Commit message' field containing 'Merge pull request #2 from Angel-luiss/feature/backend-service'. Below it is an 'Extended description' field with the text 'refactor: implementación capa Service y validaciones en productos'. At the bottom, it states 'This commit will be authored by 99922511+Angel-luiss@users.noreply.github.com.' and has two buttons: 'Confirm merge' (green) and 'Cancel' (grey). A link 'Still in progress? Convert to draft' is at the bottom right.

Pull Request cerrado y eliminación de rama

El sistema indicó que el Pull Request fue fusionado exitosamente y cerrado.

Posteriormente, se eliminó la rama feature/backend-service para mantener el repositorio limpio y organizado, siguiendo buenas prácticas de control de versiones.



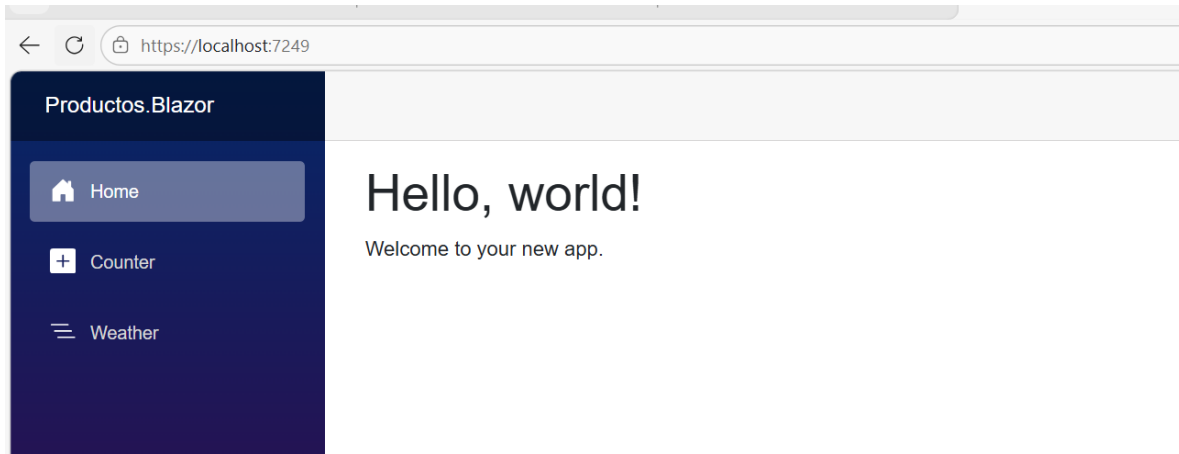
The screenshot shows the GitHub Pull Request page after a successful merge. The top section shows the commit 'refactor: implementación capa Service y validaciones en productos' (b8c88b2) merged into the 'desarrollo' branch. A red arrow points from the 'Revert' button to the 'Delete branch' button. Below this, a message states 'Pull request successfully merged and closed' and 'You're all set — the feature/backend-service branch can be safely deleted.' The 'Delete branch' button is highlighted. At the bottom, there is an 'Add a comment' section with a text area and a rich text editor toolbar.

FASE FRONTEND – CONFIGURACIÓN INICIAL

Se creó una aplicación web Blazor Server para consumir la API REST desarrollada previamente.

- `dotnet new blazor -n Productos.Blazor`

Durante la creación se seleccionó el modo **Server (Interactive Server)**.



Configuración de HttpClient para consumir la API

Se configuró el servicio HttpClient en el archivo Program.cs del proyecto Blazor para permitir la comunicación con la API.

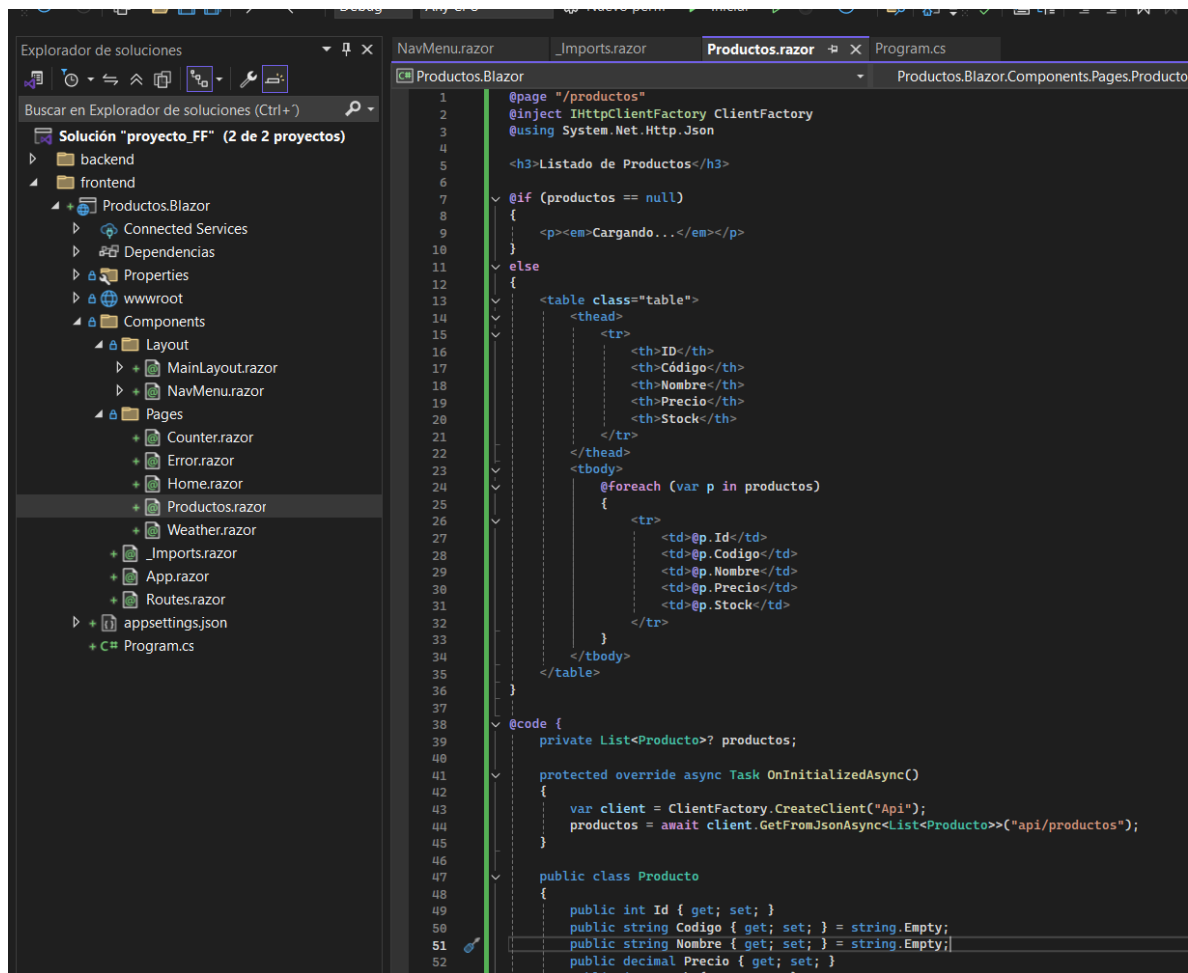
```
// Add services to the container.
builder.Services.AddRazorComponents()
    .AddInteractiveServerComponents();

// 🖱️ Agregar HttpClient para consumir la API
builder.Services.AddHttpClient("Api", client =>
{
    client.BaseAddress = new Uri("http://localhost:5116/");
});

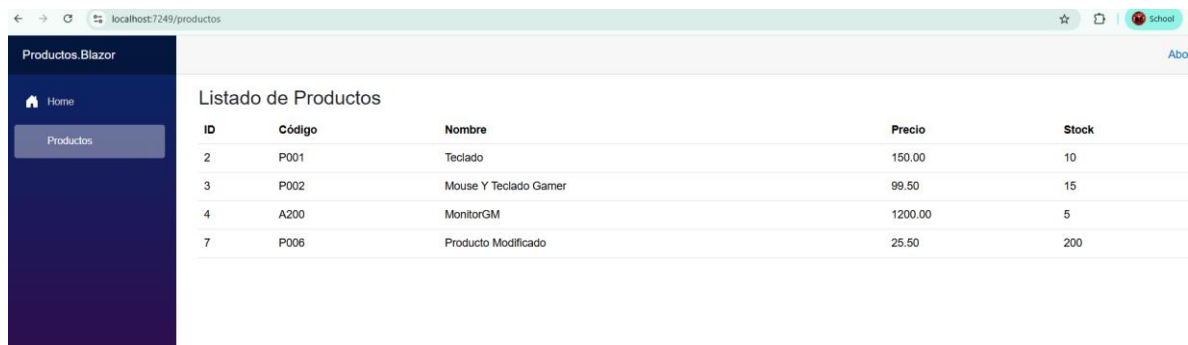
var app = builder.Build();
```

Creación de la Página Productos

Se creó el componente: **Components/Pages/Productos.razor**



```
1 @page "/productos"
2 [inject] IHttpClientFactory ClientFactory
3 @using System.Net.Http.Json
4
5 <h3>Listado de Productos</h3>
6
7 @if (productos == null)
8 {
9     <p><em>Cargando...</em></p>
10 }
11 else
12 {
13     <table class="table">
14         <thead>
15             <tr>
16                 <th>ID</th>
17                 <th>Código</th>
18                 <th>Nombre</th>
19                 <th>Precio</th>
20                 <th>Stock</th>
21             </tr>
22         </thead>
23         <tbody>
24             @foreach (var p in productos)
25             {
26                 <tr>
27                     <td>@p.Id</td>
28                     <td>@p.Código</td>
29                     <td>@p.Nombre</td>
30                     <td>@p.Precio</td>
31                     <td>@p.Stock</td>
32                 </tr>
33             }
34         </tbody>
35     </table>
36 }
37
38 @code {
39     private List<Producto>? productos;
40
41     protected override async Task OnInitializedAsync()
42     {
43         var client = ClientFactory.CreateClient("Api");
44         productos = await client.GetFromJsonAsync<List<Producto>>("api/productos");
45     }
46
47     public class Producto
48     {
49         public int Id { get; set; }
50         public string Código { get; set; } = string.Empty;
51         public string Nombre { get; set; } = string.Empty;
52         public decimal Precio { get; set; }
53     }
```



ID	Código	Nombre	Precio	Stock
2	P001	Teclado	150.00	10
3	P002	Mouse Y Teclado Gamer	99.50	15
4	A200	MonitorGM	1200.00	5
7	P006	Producto Modificado	25.50	200

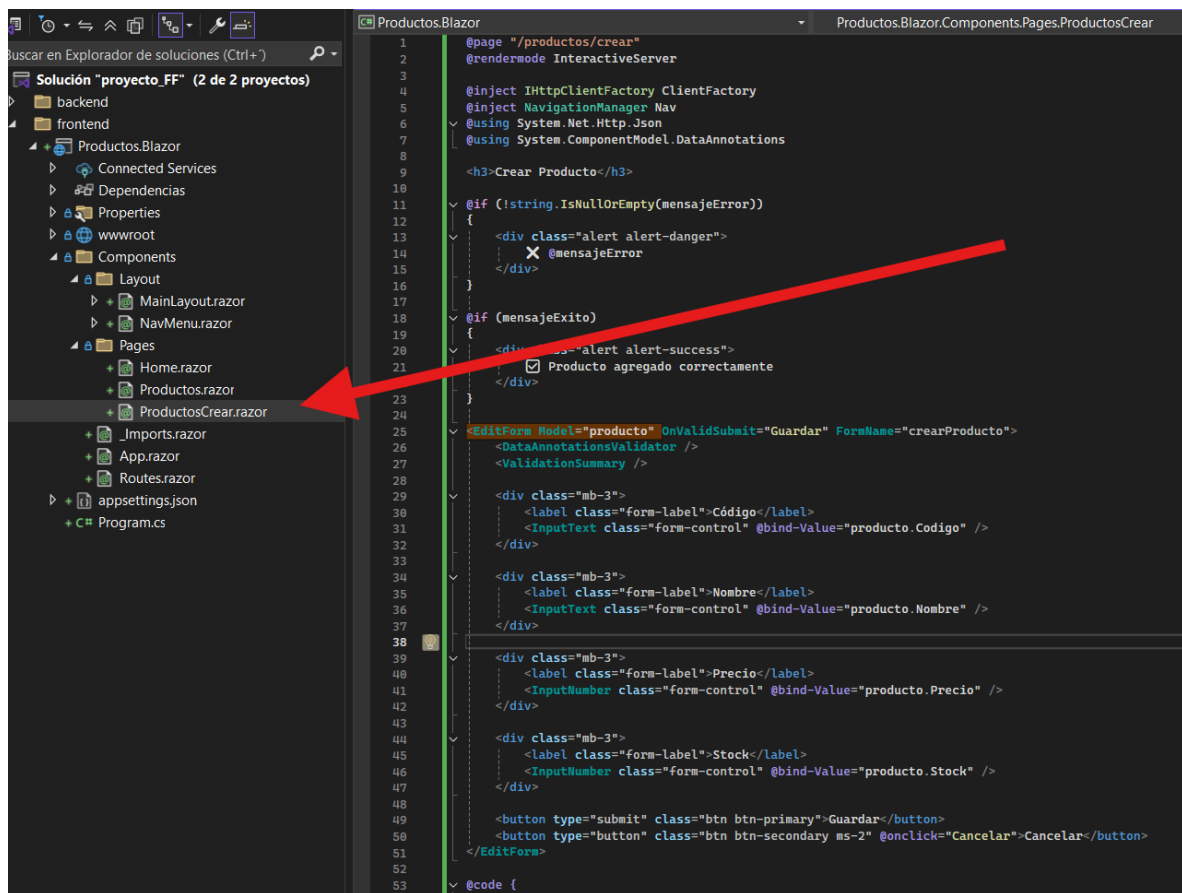
Módulo: Crear Producto (Blazor Server)

Se implementó la página Crear Producto dentro del proyecto Blazor Server, accesible mediante la ruta /productos/crear.

En la interfaz se muestra un formulario con los campos:

- Código
- Nombre
- Precio
- Stock

Cada campo cuenta con validaciones utilizando DataAnnotations, lo que permite validar automáticamente los datos antes de enviarlos al servidor.



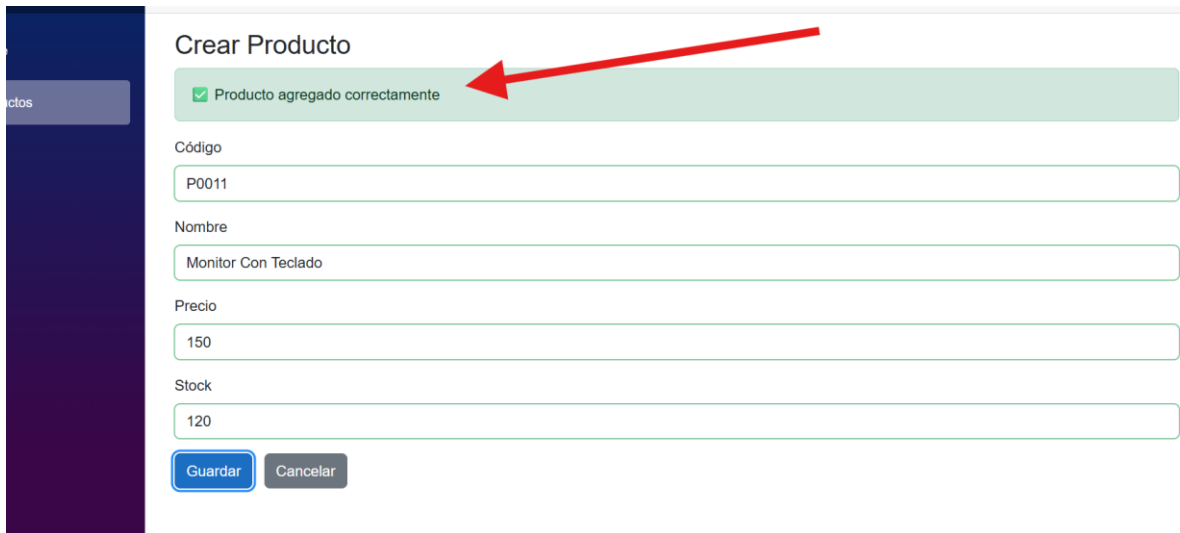
```
1 @page "/productos/crear"
2 @rendermode InteractiveServer
3
4 @inject IHttpClientFactory ClientFactory
5 @inject NavigationManager Nav
6 @using System.Net.Http.Json
7 @using System.ComponentModel.DataAnnotations
8
9 <h3>Crear Producto</h3>
10
11 <div class="alert alert-danger">
12     @mensajeError
13 </div>
14
15 <div class="alert alert-success">
16     Producto agregado correctamente
17 </div>
18
19 <EditForm Model="producto" OnValidSubmit="Guardar" FormName="crearProducto">
20     <DataAnnotationsValidator />
21     <ValidationSummary />
22
23     <div class="mb-3">
24         <label class="form-label">Código</label>
25         <input type="text" class="form-control" @bind="producto.Codigo" />
26     </div>
27
28     <div class="mb-3">
29         <label class="form-label">Nombre</label>
30         <input type="text" class="form-control" @bind="producto.Nombre" />
31     </div>
32
33     <div class="mb-3">
34         <label class="form-label">Precio</label>
35         <input type="number" class="form-control" @bind="producto.Precio" />
36     </div>
37
38     <div class="mb-3">
39         <label class="form-label">Stock</label>
40         <input type="number" class="form-control" @bind="producto.Stock" />
41     </div>
42
43     <button type="submit" class="btn btn-primary">Guardar</button>
44     <button type="button" class="btn btn-secondary ms-2" onclick="Cancelar">Cancelar</button>
45 </EditForm>
46
47 @code {
```

Caso 1 – Registro exitoso (Primera imagen)

Cuando el usuario ingresa datos válidos y el producto se guarda correctamente:

- Se muestra una notificación verde en la parte superior indicando: **“Producto agregado correctamente”**
- Después de un pequeño retraso, el sistema redirige automáticamente al listado de productos.
- En el listado ya aparece el nuevo registro insertado en la base de datos.

Esto confirma que la comunicación con la API funciona correctamente.



The screenshot shows a web form titled "Crear Producto". At the top, there is a green notification bar with a checkmark icon and the text "Producto agregado correctamente". A red arrow points to this notification. Below the notification, the form contains four input fields: "Código" with the value "P0011", "Nombre" with the value "Monitor Con Teclado", "Precio" with the value "150", and "Stock" with the value "120". At the bottom of the form, there are two buttons: "Guardar" (blue) and "Cancelar" (gray).

Caso 2 – Validaciones y errores (Segunda imagen)

Si el usuario:

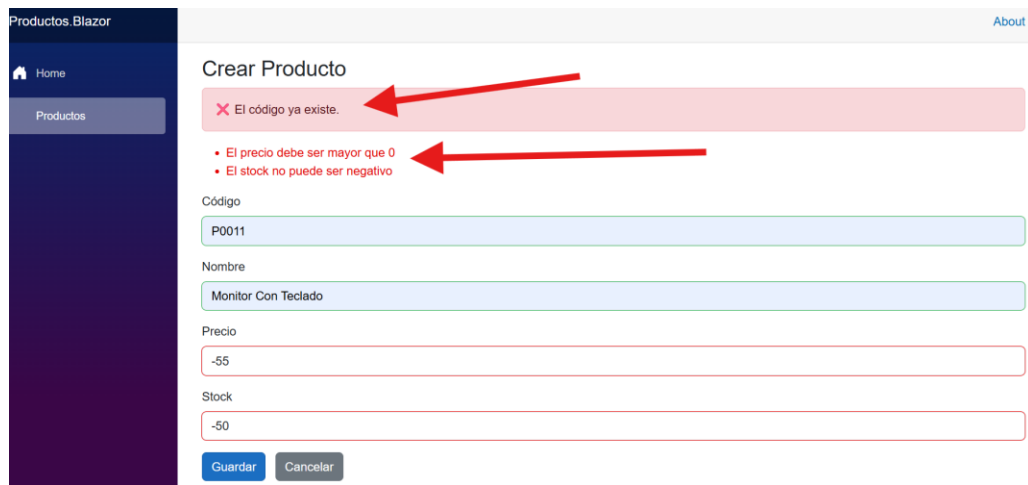
- Ingresa un precio negativo
- Ingresa stock negativo
- Intenta registrar un código que ya existe

El sistema muestra:

1. Mensajes de validación en rojo debajo del formulario (precio mayor que 0, stock no negativo).

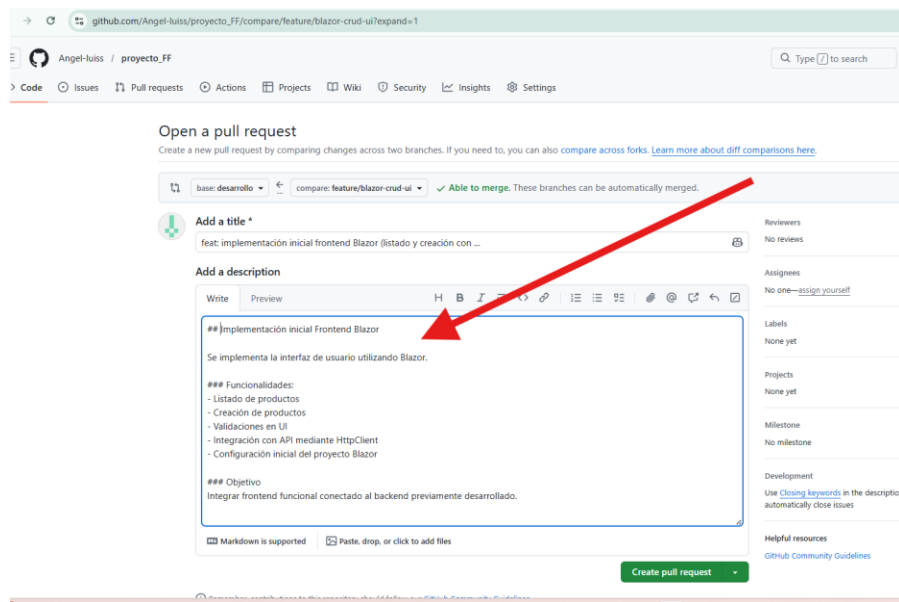
2. Un mensaje superior en rojo indicando el error devuelto por el backend, por ejemplo:
“El código ya existe.”

Esto demuestra que se implementó validación tanto en el frontend como en el backend.



Implementación y Merge de Feature Frontend Blazor

Se creó la rama feature/blazor-crud-ui a partir de desarrollo para implementar el frontend en Blazor de forma aislada.



Posteriormente se abrió un Pull Request comparando:

- **Base:** desarrollo
- **Compare:** feature/blazor-crud-ui

Conversation 0 Commits 1 Checks 0 Files changed 62

Angel-luiss commented now

Implementación inicial Frontend Blazor

Se implementa la interfaz de usuario utilizando Blazor.

Funcionalidades:

- Listado de productos
- Creación de productos
- Validaciones en UI
- Integración con API mediante HttpClient
- Configuración inicial del proyecto Blazor

Objetivo

Integrar frontend funcional conectado al backend previamente desarrollado.

feat: implementación inicial frontend Blazor (listado y creación con ... 1479129

No conflicts with base branch
Merging can be performed automatically.

Merge pull request You can also merge this with the command line. [View command line instructions.](#)

Still in progress? Convert to draft

feat: implementación inicial frontend Blazor (listado y creación con ... 1479129

Commit message

Merge pull request #3 from Angel-luiss/feature/blazor-crud-ui

Extended description

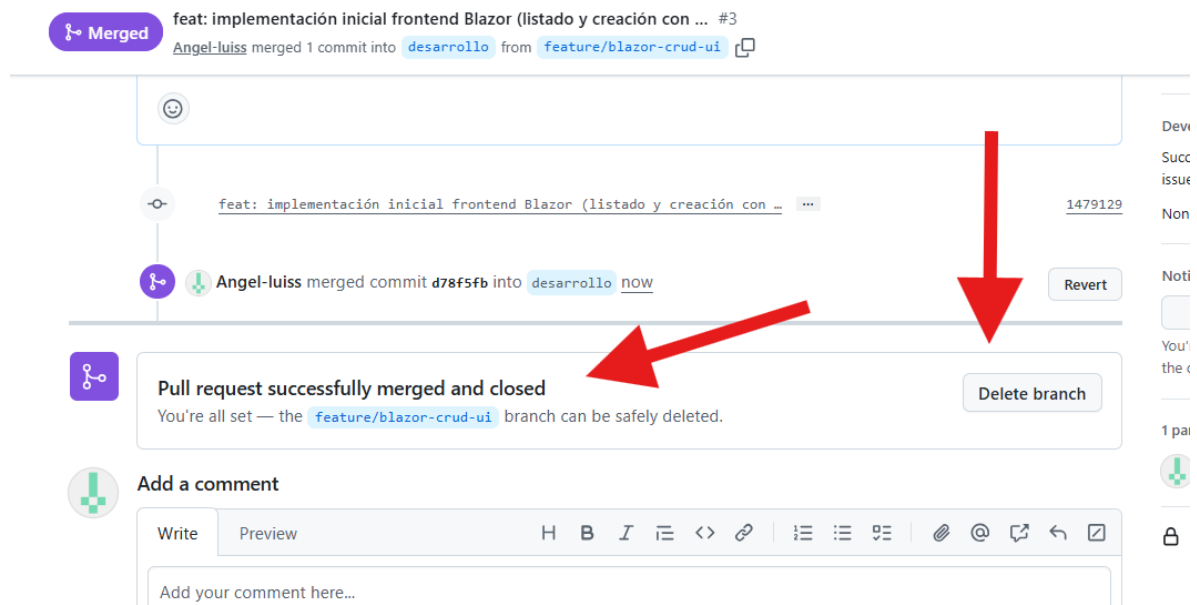
feat: implementación inicial frontend Blazor (listado y creación con ...

This commit will be authored by 99922511+Angel-luiss@users.noreply.github.com.

Confirm merge Cancel

Finalmente:

- Se realizó el **Merge Pull Request**
- La rama fue integrada correctamente en desarrollo
- Se eliminó la rama feature para mantener el repositorio limpio
- Se actualizó la rama desarrollo en local mediante git pull

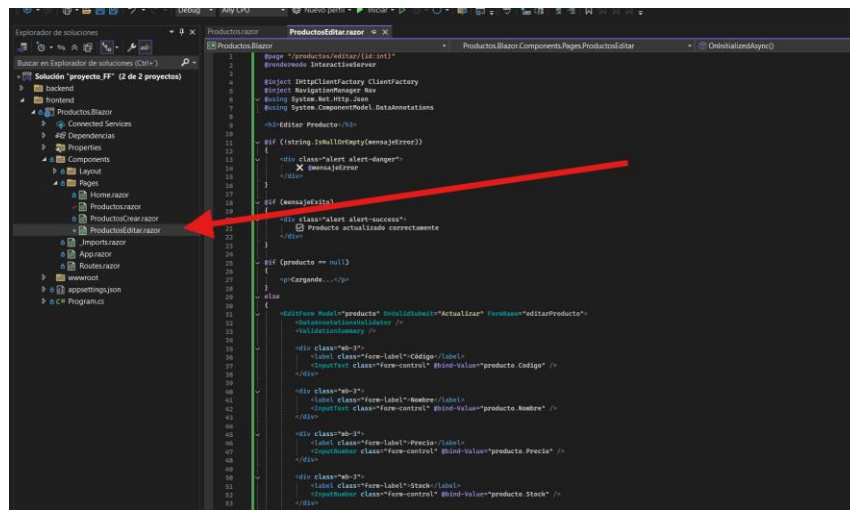


Creación del archivo ProductosEditar.razor

En esta imagen se muestra la creación del archivo ProductosEditar.razor dentro de la carpeta Pages del proyecto Blazor.

Se configuró la ruta dinámica /productos/editar/{id:int}, lo que permite recibir el identificador del producto seleccionado desde el listado.

También se implementó el formulario EditForm con validaciones y el método Actualizar() que se ejecuta al enviar el formulario.



Botón Editar en el listado

En esta imagen se observa la modificación de la tabla de productos agregando una nueva columna llamada **Acciones**.

Dentro de cada fila se añadió el botón **Editar**, el cual redirige a la página de edición enviando el Id correspondiente del producto seleccionado.

Esto permite que el usuario pueda modificar cualquier registro existente desde el listado.

Listado de Productos

[+ Nuevo Producto](#)

ID	Código	Nombre	Precio	Stock	Acciones
2	P001	Teclado	150.00	10	Editar
3	P002	Mouse Y Teclado Gamer	99.50	15	Editar
4	A200	MonitorGM	1200.00	5	Editar
7	P006	Producto Modificado	25.50	200	Editar
8	P005	MonitorGt	15.00	10	Editar
9	P0010	MonitorGta	10.00	10	Editar
10	P00101	MonitorGta	20.00	20	Editar
11	P00103	Monitor Con CPU	150.00	10	Editar
12	P0013	Monitor Con Teclado y GAME	150.00	120	Editar

Validaciones y errores en edición

En esta imagen se muestra el comportamiento del sistema cuando el usuario ingresa datos inválidos o intenta actualizar con un código duplicado.

El sistema presenta:

- Validaciones locales en color rojo (precio mayor que 0 y stock no negativo).
- Mensaje de error del backend indicando que el código ya existe.

Esto confirma que se implementó validación tanto en el frontend como en el backend.

Productos.Blazor [About](#)

Home

Productos

Editar Producto

✖ El código ya existe.

- El precio debe ser mayor que 0
- El stock no puede ser negativo

Código

Nombre

Precio

Stock

[Actualizar](#) [Cancelar](#)

Actualización exitosa

En esta imagen se observa el mensaje de confirmación en color verde indicando que el producto fue actualizado correctamente. Después de la actualización exitosa, el sistema redirige automáticamente al listado de productos, donde se reflejan los cambios realizados.

Esto confirma que la operación PUT funciona correctamente y que la información se actualiza en la base de datos.

Productos Blazor

Home

Productos

About

Editar Producto

✓ Producto actualizado correctamente

Código

P00100

Nombre

Monitor Con Teclado Gamer

Precio

150.00

Stock

120

Actualizar Cancelar

Implementación de Soft Delete en Frontend

En la interfaz se incorporó un botón de eliminación que consume el endpoint DELETE /api/Productos/{id}.

```
Program.cs ProductosController.cs Productos.razor ProductosEditar.razor ClientFactory
Productos.Blazor
56 private List<Producto>? productos;
57
58 protected override async Task OnInitializedAsync()
59 {
60     var client = ClientFactory.CreateClient("Api");
61     productos = await client.GetFromJsonAsync<List<Producto>>("api/Productos?activo=true");
62 }
63
64 private async Task Eliminar(int id)
65 {
66     var client = ClientFactory.CreateClient("Api");
67     var response = await client.DeleteAsync($"api/Productos/{id}");
68
69     if (response.IsSuccessStatusCode)
70     {
71         productos = await client.GetFromJsonAsync<List<Producto>>("api/Productos?activo=true");
72     }
73     else
74     {
75         var error = await response.Content.ReadAsStringAsync();
76         Console.WriteLine($"Error al eliminar: {error}");
77     }
78 }
79
80 public class Producto
81 {
82     public int Id { get; set; }
83     public string Codigo { get; set; } = string.Empty;
84     public string Nombre { get; set; } = string.Empty;
85     public decimal Precio { get; set; }
86     public int Stock { get; set; }
87 }
88
```

Posteriormente, se ajustó el consumo del endpoint de listado agregando el parámetro activo=true para mostrar únicamente productos activos.

Listado de Productos

+ Nuevo Producto

ID	Código	Nombre	Precio	Stock	Acciones
4	A200	MonitorGM	1200.00	5	Editar Eliminar
8	P005	MonitorGt	15.00	10	Editar Eliminar
10	P00101	MonitorGta	20.00	20	Editar Eliminar
14	P00104	Monitor Con Teclado Gamer	120.00	3	Editar Eliminar

Listado de Productos

Producto eliminado correctamente.

+ Nuevo Producto

ID	Código	Nombre	Precio	Stock	Acciones
4	A200	MonitorGM	1200.00	5	Editar Eliminar
8	P005	MonitorGt	15.00	10	Editar Eliminar
10	P00101	MonitorGta	20.00	20	Editar Eliminar

Esto permitió que los registros eliminados lógicamente (Activo = false) desaparezcan del listado sin afectar la integridad de los datos en la base de datos.

SQLQuery2.sql - D:\PCU6\admin (74)* SQLQuery1.sql - D:\PCU6\admin (53)

```

select * from dbo.Productos;
select * from dbo.__EFMigrationsHistory;

```

100 %

Results Messages

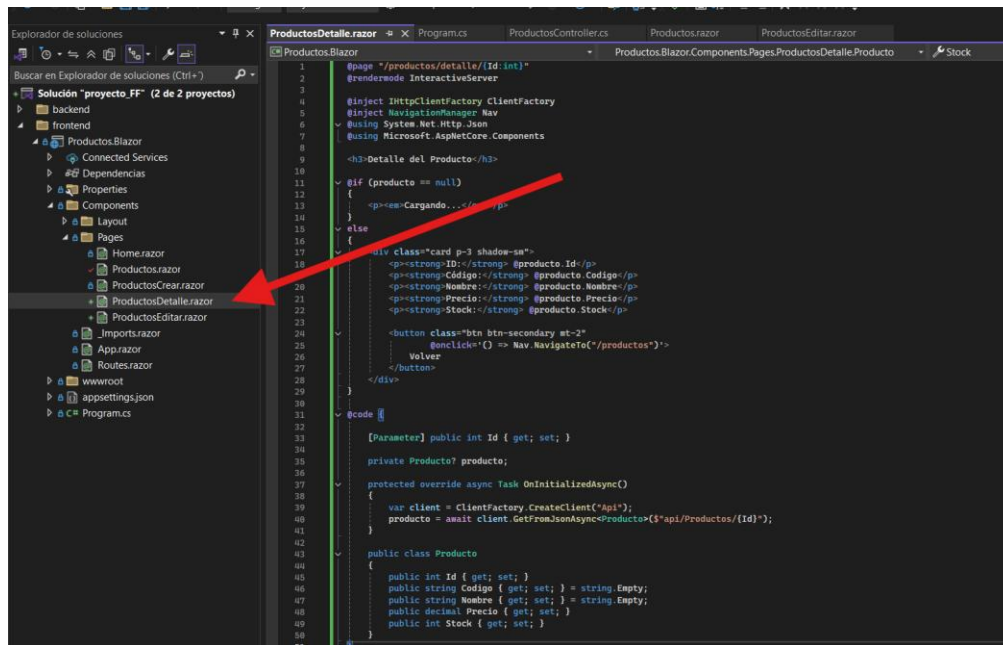
	Id	Codigo	Nombre	Precio	Stock	Activo	CreatedAt	UpdatedAt
1	2	P001	Teclado	150.00	10	0	2026-02-28 01:45:12.0005767	2026-02-28 16:08:27.6733800
2	3	P002	Mouse Y Teclado Gamer	99.50	15	0	2026-02-28 02:23:04.4128381	2026-02-28 03:46:31.9075979
3	4	A200	MonitorGM	1200.00	5	1	2026-02-28 05:52:58.3100745	NULL
4	7	P006	Producto Modificado	25.50	200	0	2026-02-28 07:25:33.7246899	2026-02-28 16:21:44.6200075
5	8	P005	MonitorGt	15.00	10	1	2026-02-28 12:13:55.1290784	NULL
6	9	P0010	MonitorGta	10.00	10	0	2026-02-28 12:16:58.8947910	2026-02-28 14:54:05.9986119
7	10	P00101	MonitorGta	20.00	20	1	2026-02-28 12:17:42.4608536	NULL
8	11	P00103	Monitor Con CPU	150.00	10	0	2026-02-28 12:17:42.4608536	2026-02-28 16:33:30.0816947
9	12	P00100	Monitor Con Teclado Gamer	150.00	120	0	2026-02-28 12:25:38.4705575	2026-02-28 16:23:15.1969647
10	13	P00102	Monitor Con Teclado Gamer	200.00	10	0	2026-02-28 16:35:50.9511587	2026-02-28 16:35:59.2351552
11	14	P00104	Monitor Con Teclado Gamer	120.00	3	0	2026-02-28 16:36:37.4833957	2026-02-28 16:37:23.2526314
12	15	P00105	Monitor Con Teclado Gamer	120.00	10	0	2026-02-28 16:37:47.9394023	2026-02-28 16:37:55.7024027

Creación de ProductosDetalle.razor

Se creó el archivo **ProductosDetalle.razor** con la ruta dinámica:

/productos/detalle/{id:int}

Este componente consume el endpoint GET /api/Productos/{id} para obtener la información de un producto específico.



```
1 @page "/productos/detalle/{id:int}"
2 #rendermode InteractiveServer
3
4 @inject HttpClientFactory ClientFactory
5 @inject NavigationManager Nav
6 #using System.Net.Http.Json
7 #using Microsoft.AspNetCore.Components
8
9 <h3>Detalle del Producto:</h3>
10
11 <div>
12     <div>
13         <p>Cargando...</p>
14     </div>
15     <div>
16         <div class="card p-3 shadow-sm">
17             <p>ID: @producto.Id</p>
18             <p>Código: @producto.Codigo</p>
19             <p>Nombre: @producto.Nombre</p>
20             <p>Precio: @producto.Precio</p>
21             <p>Stock: @producto.Stock</p>
22         </div>
23         <button class="btn btn-secondary mt-2">
24             <span>Volver</span>
25         </button>
26     </div>
27 </div>
28
29 @code {
30     [Parameter] public int Id { get; set; }
31
32     private Producto? producto;
33
34     protected override async Task OnInitializedAsync()
35     {
36         var client = ClientFactory.CreateClient("api");
37         producto = await client.GetFromJsonAsync<Producto>($"api/Productos/{Id}");
38     }
39
40     public class Producto
41     {
42         public int Id { get; set; }
43         public string Codigo { get; set; } = string.Empty;
44         public string Nombre { get; set; } = string.Empty;
45         public decimal Precio { get; set; }
46         public int Stock { get; set; }
47     }
48 }
```

Botón Ver en el listado

Se agregó el botón **Ver** en la columna Acciones del listado.

Este botón redirige a la pantalla de detalle enviando el Id del producto seleccionado.



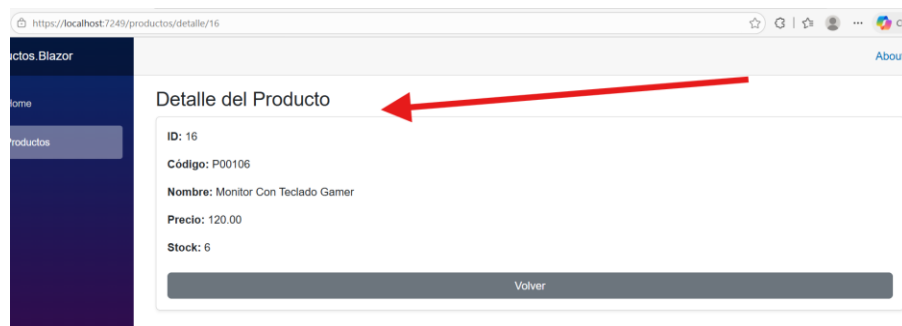
ID	Código	Nombre	Precio	Stock	Acciones
4	A200	MonitorGM	1200.00	5	Ver Editar Eliminar
8	P005	MonitorGt	15.00	10	Ver Editar Eliminar
10	P00101	MonitorGta	20.00	20	Ver Editar Eliminar

Vista Detalle funcionando

Se muestra la pantalla de detalle con los campos:

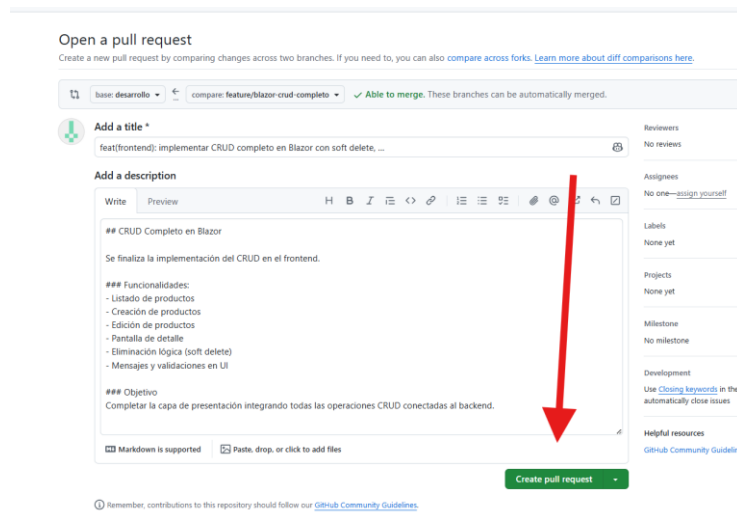
- ID
- Código
- Nombre
- Precio
- Stock

Incluye un botón **Volver** para regresar al listado.



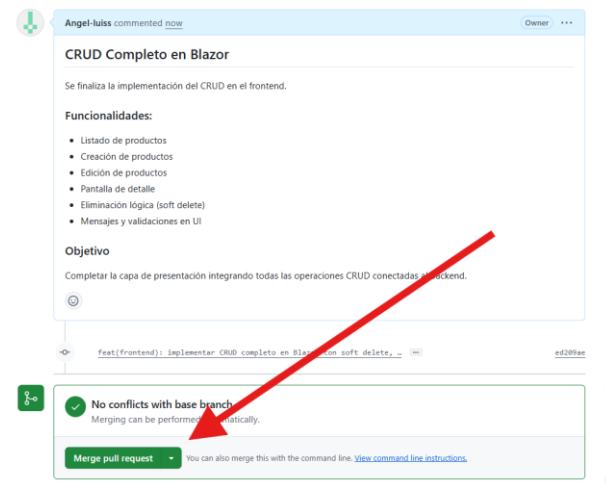
Creación del Pull Request

Se crea el Pull Request desde feature/blazor-crud-completo hacia desarrollo, documentando las funcionalidades implementadas del CRUD completo en Blazor. GitHub indica que no hay conflictos.



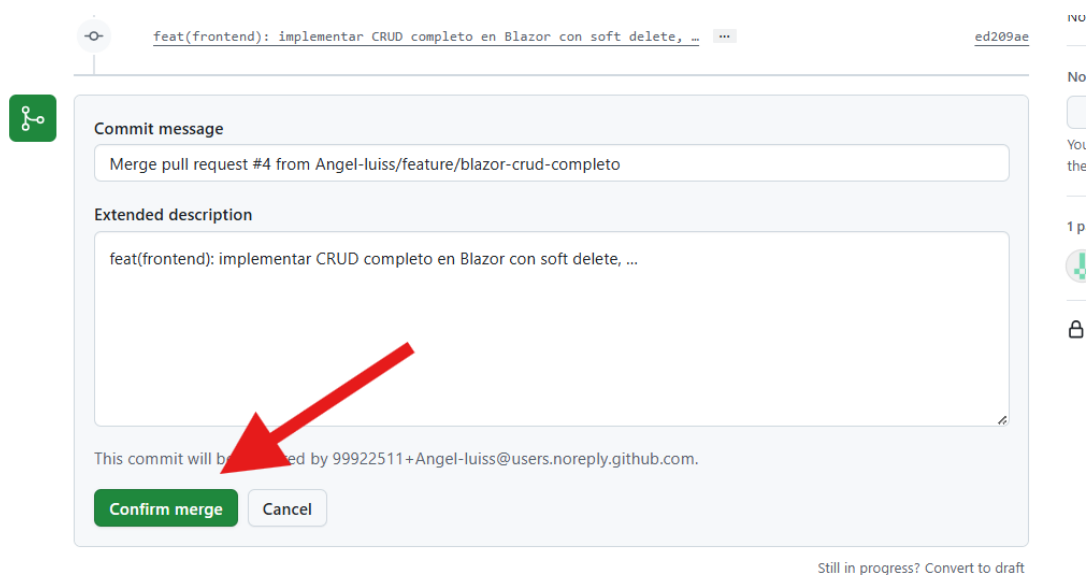
Validación de Merge

GitHub confirma que no existen conflictos con la rama base, permitiendo realizar el merge automáticamente.



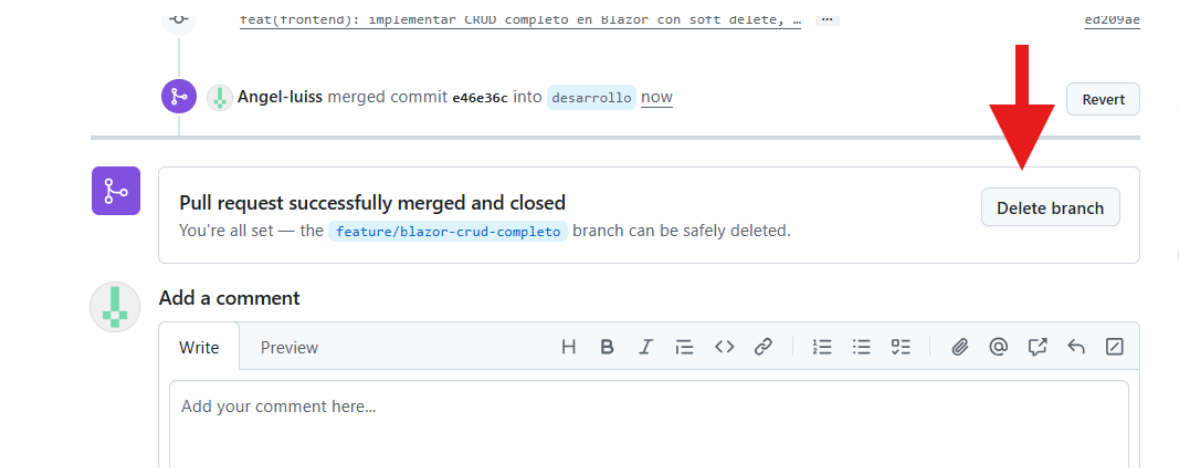
Confirmación del Merge

Se confirma el merge del Pull Request, integrando los cambios del CRUD completo en la rama desarrollo.



Merge Exitoso

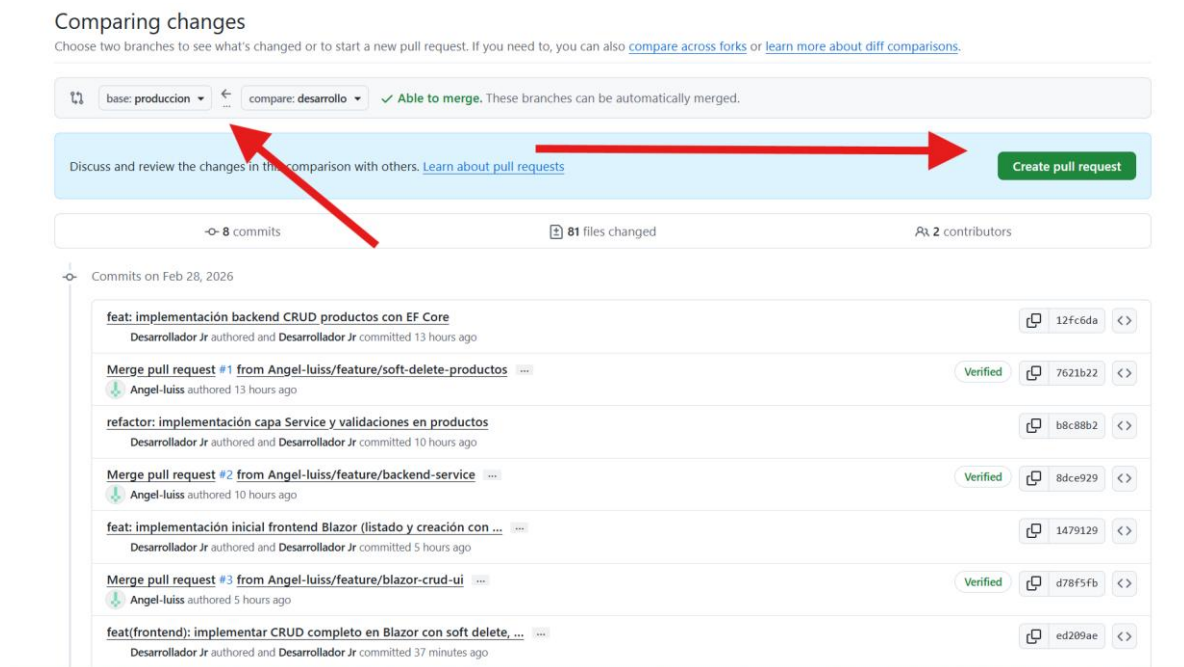
GitHub muestra que el Pull Request fue cerrado exitosamente y la rama feature puede eliminarse para mantener el repositorio limpio.



Integración Final del Sistema CRUD Full Stack en Producción

Configuración del Pull Request

Se configura el Pull Request seleccionando **base: produccion** y **compare: desarrollo**, verificando que no existan conflictos y que el merge pueda realizarse automáticamente.



Creación del Pull Request

Se define el título y la descripción del PR, detallando los componentes incluidos en el release (backend, frontend, validaciones y flujo Git), y se procede a crear el Pull Request.

Create a new pull request by comparing changes across two branches. If you need to, you can also [compare across forks](#). [Learn more about diff comparisons here](#).

base: producción ← compare: desarrollo ✓ Able to merge. These branches can be automatically merged.

Add a title *

release: integración final CRUD full stack

Add a description

Write Preview

Se integra a producción la implementación completa del sistema CRUD full stack.

Incluye:

- API ASP.NET Core con arquitectura Controller → Service → DbContext
- Validaciones de negocio (precio > 0, stock >= 0, código único)
- Soft delete mediante campo Activo
- Frontend Blazor con:
 - Listado
 - Crear
 - Editar
 - Eliminar (soft delete)
 - Detalle
- Manejo de errores y notificaciones UI

Flujo Git aplicado correctamente mediante ramas feature y merges a desarrollo.

Markdown is supported Paste, drop, or click to add files

Reviewers
No reviews

Assignees
No one—[assign yourself](#)

Labels
None yet

Projects
None yet

Milestone
No milestone

Development
Use [Closing keywords](#) in the description to automatically close issues

Helpful resources
[GitHub Community Guidelines](#)

Create pull request

Validación sin conflictos

GitHub confirma que no existen conflictos con la rama base, permitiendo realizar el merge de forma automática y segura.

Open release: integración final CRUD full stack #5

Angel-luiss wants to merge 8 commits into `producción` from `desarrollo`

Flujo Git aplicado correctamente mediante ramas feature y merges a desarrollo.

Desarrollador Jr and others added 8 commits 14 hours ago

- feat: implementación backend CRUD productos con EF Core 12fc6da
- Merge pull request #1 from Angel-luiss/feature/soft-delete-productos Verified 7621b22
- refactor: implementación capa Service y validaciones en productos b8c88b2
- Merge pull request #2 from Angel-luiss/feature/backend-service Verified 8dce929
- feat: implementación inicial frontend Blazor (listado y creación con ... 1479129
- Merge pull request #3 from Angel-luiss/feature/blazor-crud-ui Verified d78f5fb
- feat(frontend): implementar CRUD completo en Blazor con soft delete, ... ed209ae
- Merge pull request #4 from Angel-luiss/feature/blazor-crud-completo Verified e46e36c

No conflicts with base branch

Merging can be performed automatically.

Merge pull request You can also merge this with the command line. [View command line instructions](#).

Still in progress? [Convert to draft](#)

Development
Successfully merging this pull request will not create merge conflicts.

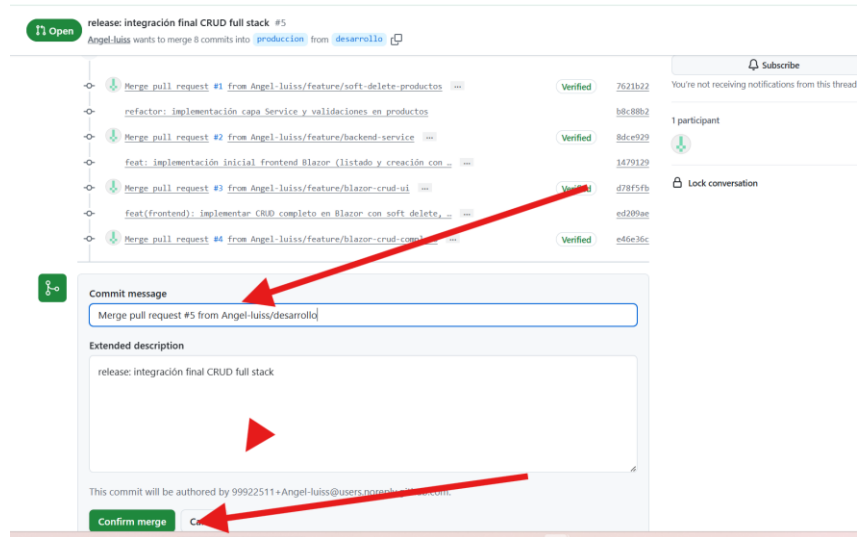
Notifications
Subscribe You're not receiving notifications from this repository.

1 participant

Lock conversation

Confirmación del Merge

Se confirma el merge del Pull Request hacia la rama **produccion**, utilizando el mensaje generado automáticamente por GitHub.



Merge Exitoso

Se muestra la confirmación de que el Pull Request fue correctamente integrado y cerrado, actualizando oficialmente la rama **produccion** con la versión final del CRUD full stack.

